Bài 2.19 – Project 1: Dự đoán giá nhà và chấm điểm tín dụng

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 3 buổi · Hình thức mở đầu: *Nhập vai nhận dự án*

🎯 Mục tiêu

Tự làm **trọn vẹn hai dự án ML** từ dữ liệu thô đến mô hình có thể bàn giao, dùng mọi kiến thức của Giai đoạn 2:

	Phần A: Giá nhà	**Phần B: Tín dụng**
Loại bài toán	Hồi quy	Phân loại (mất cân bằng)
Dữ liệu	California Housing (bài 1.12)	German Credit (bài 2.4)
Thước đo chính	MAE	Chi phí kinh doanh (+ AUC)
Bài học trọng tâm	Đặc trưng, boosting, phân tích lỗi	Ngưỡng theo chi phí, giải thích mô hình

Chia buổi:

- **Buổi 1:** Phần A (giá nhà).
- **Buổi 2:** Phần B (tín dụng).
- **Buổi 3:** Hoàn thiện báo cáo, notebook, đưa lên GitHub, luyện "bảo vệ dự án" (mục 6).

> **Cách học bài này:** Mỗi bước có **yêu cầu** và **gợi ý**. Hãy **tự làm trước**. "Lời giải tham khảo" nằm dưới các tiêu đề riêng. Trong Notion, chuyển các tiêu đề đó thành **Toggle heading** để giấu đi.

---

1. Nhập vai: Hai "khách hàng"

Bạn là ML Engineer tại một công ty tư vấn dữ liệu. Tuần này bạn nhận hai dự án.

Khách hàng 1: Công ty bất động sản

> *"Chúng tôi cần công cụ ước giá nhà theo khu vực cho đội môi giới. Yêu cầu: sai số trung bình càng nhỏ càng tốt; cho biết công cụ hay sai ở đâu để môi giới cẩn thận; và giải thích yếu tố nào ảnh hưởng giá."*

Khách hàng 2: Ngân hàng

> *"Chúng tôi muốn hệ thống chấm điểm hồ sơ vay. Theo kinh nghiệm, cho một khách xấu vay gây thiệt hại gấp 5 lần so với từ chối nhầm một khách tốt. Hãy đề xuất mô hình và ngưỡng duyệt hồ sơ giúp giảm tổng thiệt hại. Chúng tôi cũng cần biết hệ thống dựa vào yếu tố gì để quyết định."*

---

2. Yêu cầu chung và tiêu chí chấm

Quy tắc bắt buộc:

1. Chia train/test **ngay từ đầu**. Test chỉ mở **một lần** ở bước đánh giá cuối.
2. Mọi lựa chọn (đặc trưng, mô hình, siêu tham số, ngưỡng) dựa trên **cross-validation có xáo trộn** trên train.
3. Mọi bước "học" từ dữ liệu nằm trong **Pipeline**.
4. Luôn so với **mốc** (baseline).
5. Notebook chạy lại từ đầu đến cuối không lỗi (`Runtime → Restart and run all`).

Tiêu chí tự chấm (mỗi phần):

Tiêu chí	Đạt khi
Quy trình	Không rò rỉ: chia trước, pipeline, CV trên train, test dùng một lần
So sánh	Ít nhất 3 mô hình + mốc, so bằng CV, có trung bình ± độ lệch chuẩn
Cải thiện	Có ít nhất 1 cải tiến được **kiểm chứng** (đặc trưng hoặc tinh chỉnh)
Đánh giá	Kết quả test bằng chỉ số phù hợp bài toán, kèm diễn giải bằng lời
Phân tích lỗi / giải thích	Mô hình sai ở đâu (A); dựa vào yếu tố gì (A, B)
Bàn giao	Lưu mô hình bằng joblib, có ví dụ dự đoán cho dữ liệu mới
Báo cáo	Người không chuyên đọc hiểu; nêu rõ **giới hạn** của mô hình

---

PHẦN A: DỰ ĐOÁN GIÁ NHÀ

A1. Chuẩn bị và chia dữ liệu

Yêu cầu: đọc dữ liệu, đổi tên cột, bỏ các khu bị giới hạn trần giá, đổi giá sang nghìn USD, chia 80/20 với `random_state=42`.

Gợi ý: bài 1.12 (bước 1–3, 8), bài 2.16 (mục 3.1).

Lời giải tham khảo A1

```
import time
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

url = "https://download.mlcc.google.com/mledu-
datasets/california_housing_train.csv"
raw = pd.read_csv(url)
raw.columns = ["kinh_do", "vi_do", "tuoi_nha", "tong_phong", "tong_phong_ngu",
               "dan_so", "so_ho", "thu_nhap", "gia_nha"]
raw = raw[raw["gia_nha"] < 500001].reset_index(drop=True)      # bỏ giới hạn trần
(ghi vào báo cáo!)
raw["gia_nha"] = raw["gia_nha"] / 1000
train_raw, test_raw = train_test_split(raw, test_size=0.2, random_state=42)   #
```

```
tách test NGAY
print(train_raw.shape, test_raw.shape)
```

Kết quả mẫu:

```
(12948, 9) (3238, 9)
```

A2. Tạo đặc trưng

Yêu cầu: viết hàm `them_dac_trung(d)` tạo các đặc trưng đã kiểm chứng ở bài 2.16 (tỉ lệ, khoảng cách tới LA/SF, log). Áp dụng cho cả train và test.

Lời giải tham khảo A2

```
def them_dac_trung(d):
    """Đặc trưng không 'học' từ dữ liệu → áp dụng an toàn cho cả train và test."""
    d = d.copy()
    d["phong_moi_ho"] = d["tong_phong"] / d["so_ho"]
    d["ti_le_phong_ngu"] = d["tong_phong_ngu"] / d["tong_phong"]
    d["nguoi_moi_ho"] = d["dan_so"] / d["so_ho"]
    km = lambda v1, k1, v2, k2: np.sqrt(((k1 - k2) * 111 * np.cos(np.radians(v1)))
** 2 + ((v1 - v2) * 111) ** 2)
    d["km_tp_lon"] = np.minimum(km(d["vi_do"], d["kinh_do"], 34.05, -118.24),
                                km(d["vi_do"], d["kinh_do"], 37.77, -122.42))
    for c in ["phong_moi_ho", "nguoi_moi_ho", "km_tp_lon", "thu_nhap"]:
        d["log_" + c] = np.log1p(d[c])
    return d

DAC_TRUNG = ["kinh_do", "vi_do", "tuoi_nha", "tong_phong", "tong_phong_ngu",
"dan_so", "so_ho", "thu_nhap",
             "ti_le_phong_ngu", "log_phong_moi_ho", "log_nguoi_moi_ho",
"log_km_tp_lon", "log_thu_nhap"]
train, test = them_dac_trung(train_raw), them_dac_trung(test_raw)
Xtr, ytr = train[DAC_TRUNG], train["gia_nha"]
Xte, yte = test[DAC_TRUNG], test["gia_nha"]
```

A3. So sánh mô hình bằng cross-validation

Yêu cầu: so sánh ít nhất: mốc, Linear Regression (có chuẩn hóa), Random Forest, LightGBM. Dùng `KFold(5, shuffle=True, random_state=42)` và MAE.

Lời giải tham khảo A3

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.dummy import DummyRegressor
import lightgbm as lgb

cv = KFold(n_splits=5, shuffle=True, random_state=42)
ung_vien = {
    "Mốc (trung bình)": DummyRegressor(),
    "Linear Regression": make_pipeline(StandardScaler(), LinearRegression()),
    "Random Forest": RandomForestRegressor(n_estimators=300, min_samples_leaf=2,
random_state=42, n_jobs=-1),
    "LightGBM": lgb.LGBMRegressor(n_estimators=600, learning_rate=0.05,
random_state=42, verbose=-1),
}
for ten, m in ung_vien.items():
    s = -cross_val_score(m, Xtr, ytr, cv=cv, scoring="neg_mean_absolute_error")
    print(f"{ten:<18}: MAE CV = {s.mean():5.1f} ± {s.std():.1f}")
```

Kết quả mẫu (khoảng 1 phút):

```
Mốc (trung bình)  : MAE CV =  78.8 ± 1.4
Linear Regression : MAE CV =  42.2 ± 0.8
Random Forest     : MAE CV =  30.3 ± 0.7
LightGBM          : MAE CV =  27.9 ± 0.7
```

LightGBM tốt nhất, và chênh lệch với Random Forest (2,4) **lớn hơn nhiều** độ dao động (±0,7): có thể tin là thật.

A4. Tinh chỉnh

Yêu cầu: tinh chỉnh LightGBM bằng `RandomizedSearchCV` (20–30 lần thử). So với mặc định.

Lời giải tham khảo A4

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint, uniform

pham_vi = {"learning_rate": loguniform(0.01, 0.2), "num_leaves": randint(15, 150),
           "min_child_samples": randint(5, 60), "subsample": uniform(0.6, 0.4),
           "colsample_bytree": uniform(0.5, 0.5), "reg_lambda": loguniform(1e-3,
10)}
rs = RandomizedSearchCV(lgb.LGBMRegressor(n_estimators=1000, subsample_freq=1,
random_state=42, verbose=-1),
                        pham_vi, n_iter=25, cv=cv,
scoring="neg_mean_absolute_error",
                        random_state=42, n_jobs=-1).fit(Xtr, ytr)
print({k: (float(round(v, 4)) if isinstance(v, float) else v) for k, v in
```

```
rs.best_params_.items()})
print("MAE CV tốt nhất:", round(-rs.best_score_, 2))
```

Kết quả mẫu (khoảng 2–3 phút):

```
{'colsample_bytree': 0.5078, 'learning_rate': 0.0356, 'min_child_samples': 5,
'num_leaves': 77, 'reg_lambda': 1.8275, 'subsample': 0.8827}
MAE CV tốt nhất: 27.31
```

Cải thiện từ 27,9 xuống 27,3, tức 0,6 nghìn USD, xấp xỉ bằng độ dao động giữa các fold (±0,7). Có ích nhưng không lớn: ghi trung thực như vậy vào báo cáo, đừng thổi phồng.

A5. Đánh giá trên test và phân tích lỗi

Yêu cầu:

1. Tính MAE, R^2, MAPE trên test (**một lần**).
2. Mô hình sai nhiều nhất ở **vùng nào** và **mức giá nào**? Có xu hướng đoán cao hay thấp?
3. Yếu tố nào quan trọng nhất (permutation importance)?

Lời giải tham khảo A5

```
from sklearn.metrics import mean_absolute_error, r2_score,
mean_absolute_percentage_error
from sklearn.inspection import permutation_importance

p = rs.predict(Xte)
print(f"TEST: MAE = {mean_absolute_error(yte, p):.1f} | R² = {r2_score(yte,
p):.3f} | MAPE = {mean_absolute_percentage_error(yte, p):.1%}")
sai = np.abs(yte - p)
print("Trung vị sai số:", round(np.median(sai), 1), "| 10% tệ nhất sai trên:",
round(np.percentile(sai, 90), 1))
```

Kết quả mẫu:

```
TEST: MAE = 26.4 | R² = 0.832 | MAPE = 15.5%
Trung vị sai số: 17.3 | 10% tệ nhất sai trên: 60.6
```

MAE test (26,4) rất gần MAE CV (27,3): cross-validation đã ước lượng chính xác. **Không có dấu hiệu rò rỉ hay overfit.**

```
ket = test.assign(du_doan=p, sai=sai)
ket["nhom_km"] = pd.cut(ket["km_tp_lon"], [0, 25, 100, 1000], labels=["<25 km",
"25-100 km", ">100 km"])
```

```
print(ket.groupby("nhom_km", observed=True).agg(
    so_khu=("sai", "size"), gia_tb=("gia_nha", "mean"), mae=("sai", "mean"),
    mape=("sai", lambda s: (s / ket.loc[s.index, "gia_nha"]).mean())).round(3))

ket["nhom_gia"] = pd.cut(ket["gia_nha"], [0, 100, 200, 300, 500], labels=["<100",
"100-200", "200-300", "300-500"])
print(ket.groupby("nhom_gia", observed=True).agg(
    so_khu=("sai", "size"), mae=("sai", "mean"),
    lech=("du_doan", lambda s: (s - ket.loc[s.index,
"gia_nha"]).mean())).round(1))   # lech > 0: đoán CAO hơn thật
```

Kết quả mẫu:

```
           so_khu   gia_tb     mae   mape
nhom_km
<25 km        861  223.639  29.305  0.131
25-100 km    1187  222.581  28.033  0.141
>100 km      1188  135.498  22.707  0.185
          so_khu   mae  lech
nhom_gia
<100         612  16.5  13.3
100-200     1361  20.4   8.2
200-300      822  29.0  -1.3
300-500      443  54.1 -39.3
```

Phân tích lỗi:

- **Vùng xa thành phố (>100 km):** sai số tuyệt đối nhỏ nhất (22,7) nhưng sai số **tương đối** lớn nhất (18,5%), vì nhà ở đó rẻ.
- **Theo mức giá:** mô hình đoán **cao hơn** thật với nhà rẻ (+13,3) và đoán **thấp hơn** thật rất nhiều với nhà đắt (−39,3). Đây là hiện tượng **"kéo về trung bình"**: khi không chắc chắn, mô hình có xu hướng đoán gần mức trung bình. Với nhà 300–500 nghìn USD, sai số trung bình **54.000 USD**: môi giới cần đặc biệt cẩn thận ở phân khúc cao cấp.

```
pi = permutation_importance(rs.best_estimator_, Xte, yte,
scoring="neg_mean_absolute_error",
                            n_repeats=5, random_state=42, n_jobs=-1)
print(pd.Series(pi.importances_mean,
index=DAC_TRUNG).sort_values(ascending=False).head(6).round(1))
```

Kết quả mẫu:

```
vi_do               36.1
kinh_do             32.5
log_km_tp_lon       18.7
thu_nhap            10.5
log_nguoi_moi_ho     8.7
```

```
log_thu_nhap         3.7
dtype: float64
```

Xáo trộn vĩ độ → MAE tăng thêm 36 nghìn USD. **Vị trí** là yếu tố số 1 (vĩ độ, kinh độ, khoảng cách tới thành phố lớn), rồi đến **thu nhập**. (Thu nhập có 2 cột: `thu_nhap` và `log_thu_nhap` gần như trùng thông tin, nên permutation importance của từng cột bị "chia nhỏ", bài 2.10.)

A6. Đóng gói mô hình

Yêu cầu: tạo một pipeline nhận **dữ liệu thô** (9 cột gốc) và trả về giá dự đoán. Lưu bằng joblib.

Lời giải tham khảo A6

```
import joblib
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer

def tao_dac_trung(d):
    """Bước 'không học': tạo đặc trưng và chọn đúng các cột mô hình cần."""
    return them_dac_trung(d)[DAC_TRUNG]

mo_hinh_nha = Pipeline([("dac_trung", FunctionTransformer(tao_dac_trung)),   # bọc
hàm thành một bước pipeline
                        ("lgbm", rs.best_estimator_)])
mau = test_raw.drop(columns="gia_nha").iloc[:2]        # dữ liệu THÔ, như người
dùng nhập vào
print(mo_hinh_nha.predict(mau).round(1), test_raw["gia_nha"].iloc[:2].values)
joblib.dump(mo_hinh_nha, "mo_hinh_gia_nha.joblib")
```

Kết quả mẫu:

```
[ 52.  179.4] [ 52.6 178.4]
```

> ⚠️ File joblib lưu hàm `tao_dac_trung` theo **tên**, không lưu nội dung hàm. Khi tải lại ở chương trình khác, hàm (và `them_dac_trung`, `DAC_TRUNG`) phải được định nghĩa hoặc import trước. Ở Phần C, ta sẽ đặt chúng trong một file `.py` riêng. Đây cũng là lý do **không dùng** `lambda` trong pipeline cần lưu.

PHẦN B: CHẤM ĐIỂM TÍN DỤNG

B1. Chuẩn bị, chia dữ liệu, dựng tiền xử lý

Yêu cầu: tải dữ liệu, tạo y (1 = vỡ nợ), chia 80/20 **có stratify**, dựng `ColumnTransformer` cho cột số và cột chữ.

Gợi ý: bài 2.17 mục 3.

Lời giải tham khảo B1

```
from sklearn.datasets import fetch_openml
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder

credit = fetch_openml("credit-g", version=1, as_frame=True).frame
X = credit.drop(columns="class")
y = (credit["class"] == "bad").astype(int)
Xtr_c, Xte_c, ytr_c, yte_c = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

cot_so = X.select_dtypes("number").columns.tolist()
cot_chu = X.select_dtypes("category").columns.tolist()
tien_xu_ly = ColumnTransformer([
    ("so", Pipeline([("dien", SimpleImputer(strategy="median")), ("chuan_hoa",
StandardScaler())]), cot_so),
    ("chu", Pipeline([("dien", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
cot_chu),
])
```

B2. So sánh mô hình

Yêu cầu: so sánh ít nhất 3 mô hình bằng AUC với `StratifiedKFold(5, shuffle=True, random_state=42)`. Thử thêm Logistic Regression với regularization mạnh hơn (C = 0,1).

Lời giải tham khảo B2

```
from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

cv_c = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
ung_vien_c = {
    "Logistic (C=1)":   Pipeline([("tx", tien_xu_ly), ("mh",
LogisticRegression(max_iter=2000))]),
    "Logistic (C=0.1)": Pipeline([("tx", tien_xu_ly), ("mh",
LogisticRegression(C=0.1, max_iter=2000))]),
    "Random Forest":    Pipeline([("tx", tien_xu_ly), ("mh",
RandomForestClassifier(n_estimators=500, min_samples_leaf=3,

random_state=42, n_jobs=-1))]),
    "LightGBM":         lgb.LGBMClassifier(n_estimators=300, learning_rate=0.03,
num_leaves=15, subsample=0.8,
                                           subsample_freq=1, colsample_bytree=0.8,
random_state=42, verbose=-1),
}
for ten, m in ung_vien_c.items():
```

```
    s = cross_val_score(m, Xtr_c, ytr_c, cv=cv_c, scoring="roc_auc")
    print(f"{ten:<17}: AUC CV = {s.mean():.3f} ± {s.std():.3f}")
```

Kết quả mẫu:

```
Logistic (C=1)   : AUC CV = 0.771 ± 0.053
Logistic (C=0.1) : AUC CV = 0.784 ± 0.050
Random Forest    : AUC CV = 0.794 ± 0.040
LightGBM         : AUC CV = 0.783 ± 0.034
```

- Regularization mạnh hơn giúp Logistic tăng từ 0,771 lên 0,784 (bài 2.17 đã thấy Logistic C=1 hơi overfit).
- Random Forest cao nhất (0,794). Chênh lệch giữa các mô hình nhỏ hơn độ dao động: **chưa thể khẳng định** chắc chắn. Chọn Random Forest vì vừa cao nhất vừa ổn định. Logistic (C = 0,1) là lựa chọn thay thế tốt nếu ngân hàng cần **giải thích từng hệ số**.

B3. Chọn ngưỡng theo chi phí (trên train, bằng CV)

Yêu cầu: chi phí = 5 × FN + 1 × FP. Tìm ngưỡng làm chi phí nhỏ nhất, **không dùng tập test**.

Gợi ý: `cross_val_predict(..., method="predict_proba")` cho xác suất "như trên dữ liệu mới" của **mọi** khách trong train: mỗi khách được dự đoán bởi mô hình chưa từng thấy họ.

Lời giải tham khảo B3

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import confusion_matrix

chon = ung_vien_c["Random Forest"]
p_cv = cross_val_predict(chon, Xtr_c, ytr_c, cv=cv_c, method="predict_proba")[:,
1]
nguong_list = np.round(np.arange(0.05, 0.80, 0.05), 2)
chi_phi = []
for t in nguong_list:
    tn, fp, fn, tp = confusion_matrix(ytr_c, (p_cv >= t).astype(int)).ravel()
    chi_phi.append(5 * fn + fp)
bang = pd.DataFrame({"nguong": nguong_list, "chi_phi_cv": chi_phi})
print(bang.T.to_string())
nguong_tot = float(bang.loc[bang["chi_phi_cv"].idxmin(), "nguong"])
print("Ngưỡng tốt nhất:", nguong_tot)
```

Kết quả mẫu:

```
                0      1       2      3       4      5       6      7       8
9       10      11       12      13       14
nguong        0.05    0.1    0.15    0.2    0.25    0.3    0.35    0.4    0.45
```

```
0.5    0.55     0.6     0.65     0.7     0.75
chi_phi_cv  542.00  481.0  423.00  429.0  407.00  468.0  516.00  608.0  758.00
822.0  994.00  1093.0  1158.00  1191.0  1195.00
Ngưỡng tốt nhất: 0.25
```

Chi phí thấp nhất ở ngưỡng **0,25** (407), gần một nửa so với ngưỡng mặc định 0,5 (822).

B4. Đánh giá trên test

Yêu cầu: huấn luyện mô hình cuối trên toàn bộ train, đánh giá trên test với ngưỡng 0,5 và ngưỡng đã chọn. So sánh với hai chiến lược "ngây thơ": **cho vay tất cả** và **từ chối tất cả**.

Lời giải tham khảo B4

```
from sklearn.metrics import roc_auc_score, recall_score, precision_score

chon.fit(Xtr_c, ytr_c)
pt = chon.predict_proba(Xte_c)[:, 1]
print("AUC test:", round(roc_auc_score(yte_c, pt), 3))
for t in [0.5, nguong_tot]:
    d = (pt >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(yte_c, d).ravel()
    print(f"Ngưỡng {t}: TP={tp} FP={fp} FN={fn} TN={tn} | Recall=
{recall_score(yte_c, d):.2f}"
          f" Precision={precision_score(yte_c, d):.2f} | Chi phí={5 * fn + fp} |
Từ chối {d.mean():.0%}")
print("Cho vay tất cả: chi phí", 5 * yte_c.sum())                # mọi khách xấu
đều lọt
print("Từ chối tất cả: chi phí", (yte_c == 0).sum())             # mọi khách tốt
đều bị từ chối
```

Kết quả mẫu:

```
AUC test: 0.801
Ngưỡng 0.5: TP=24 FP=10 FN=36 TN=130 | Recall=0.40 Precision=0.71 | Chi phí=190 |
Từ chối 17%
Ngưỡng 0.25: TP=52 FP=64 FN=8 TN=76 | Recall=0.87 Precision=0.45 | Chi phí=104 |
Từ chối 58%
Cho vay tất cả: chi phí 300
Từ chối tất cả: chi phí 140
```

Diễn giải trung thực, phần quan trọng nhất của dự án:

Chiến lược	Chi phí	Nhận xét
Cho vay tất cả	300	Tệ nhất
Mô hình, ngưỡng 0,5	190	**Còn tệ hơn từ chối tất cả!**

Chiến lược	Chi phí	Nhận xét
Từ chối tất cả	140	Không kinh doanh được gì
Mô hình, ngưỡng 0,25	**104**	Tốt nhất: giảm 65% so với cho vay tất cả

- Với ngưỡng mặc định, mô hình **thua cả chiến lược "đóng cửa"**. Đây là bằng chứng rõ nhất cho bài 2.5: ngưỡng phải theo chi phí.
- Với ngưỡng 0,25: bắt được **87%** khách xấu, nhưng **từ chối 58%** số hồ sơ, trong đó 64 / 140 khách tốt (46%) bị từ chối oan.
- **Giới hạn của ma trận chi phí:** nó chưa tính **lợi nhuận** từ khách tốt được cho vay, cũng chưa tính khách bị từ chối sẽ sang ngân hàng khác. Một ngân hàng thật sẽ không chấp nhận từ chối 58% hồ sơ. **Đề xuất:** dùng 3 vùng thay vì 2: điểm thấp → duyệt tự động; điểm cao → từ chối; vùng giữa → **chuyển nhân viên thẩm định**. Mô hình hỗ trợ con người, không thay thế.

B5. Giải thích mô hình

Yêu cầu: dùng permutation importance (AUC) trên test, lấy 6 yếu tố quan trọng nhất.

Lời giải tham khảo B5

```
pi_c = permutation_importance(chon, Xte_c, yte_c, scoring="roc_auc", n_repeats=10,
random_state=42, n_jobs=-1)
print(pd.Series(pi_c.importances_mean,
index=X.columns).sort_values(ascending=False).head(6).round(3))
```

Kết quả mẫu:

```
checking_status    0.134
credit_history     0.020
duration           0.019
credit_amount      0.019
purpose            0.012
age                0.011
dtype: float64
```

Vì permutation importance áp dụng trên **dữ liệu thô** đưa vào pipeline, mỗi cột chữ được đo **trọn vẹn** (không bị chia nhỏ thành nhiều cột one-hot như ở bài 2.10). **Tình trạng tài khoản thanh toán** áp đảo (AUC giảm 0,134 khi xáo trộn), sau đó là lịch sử tín dụng, thời hạn vay, số tiền vay.

> ⚠️ **Lưu ý đạo đức:** dữ liệu có cột `personal_status` (gộp giới tính và tình trạng hôn nhân) và `foreign_worker` (lao động nước ngoài). Dùng chúng để từ chối cho vay có thể là **phân biệt đối xử** và vi phạm pháp luật ở nhiều nước. Bài tập mở rộng: bỏ hai cột này, đo lại AUC và chi phí. Nếu mô hình gần như không kém đi, **không có lý do gì** để giữ chúng.

B6. Đóng gói

```
joblib.dump({"mo_hinh": chon, "nguong": nguong_tot}, "mo_hinh_tin_dung.joblib")
# lưu cả ngưỡng!
kh = Xte_c.iloc[[0, 1]]
print(chon.predict_proba(kh)[:, 1].round(3), yte_c.iloc[:2].values)
```

Kết quả mẫu:

```
[0.272 0.277] [0 0]
```

Hai khách này có điểm rủi ro 0,27–0,28, **vượt** ngưỡng 0,25 → bị từ chối, dù thật ra cả hai đều là khách tốt (nhãn 0). Đây chính là "vùng xám" nên chuyển nhân viên thẩm định.

---

5. Báo cáo mẫu (dán vào Notion)

```
BÁO CÁO PROJECT 1

——— A. CÔNG CỤ ƯỚC GIÁ NHÀ ———
TÓM TẮT: Mô hình LightGBM ước giá nhà theo khu vực, sai số trung bình 26.400 USD
(khoảng 15% giá nhà), giải thích 83% sự khác biệt giá giữa các khu.

KẾT QUẢ (tập test, 3.238 khu chưa từng thấy):
- MAE 26,4 nghìn USD | R² 0,832 | MAPE 15,5%
- So với mốc "đoán giá trung bình": sai số giảm 66% (78,8 → 26,4)
- Một nửa số khu sai dưới 17.300 USD; 10% khu sai trên 60.600 USD

KHI NÀO CẦN CẨN THẬN:
- Nhà trên 300.000 USD: mô hình thường đoán THẤP hơn thật khoảng 39.000 USD
- Nhà dưới 100.000 USD: thường đoán CAO hơn thật khoảng 13.000 USD
- KHÔNG dùng cho khu trên 500.000 USD (dữ liệu huấn luyện không có)
- Dữ liệu năm 1990: cần huấn luyện lại với dữ liệu mới trước khi dùng thật

YẾU TỐ ẢNH HƯỞNG: vị trí (vĩ độ, kinh độ, khoảng cách tới LA/SF) > thu nhập khu
vực
> mật độ người mỗi hộ

——— B. CHẤM ĐIỂM TÍN DỤNG ———
TÓM TẮT: Mô hình Random Forest với ngưỡng 0,25 giảm 65% thiệt hại so với cho vay
tất cả, bắt được 87% khách rủi ro.

KẾT QUẢ (tập test, 200 hồ sơ): AUC 0,801
- Ngưỡng 0,25: chi phí 104 (cho vay tất cả: 300; từ chối tất cả: 140)
- Đánh đổi: từ chối 58% hồ sơ, trong đó 46% khách tốt bị từ chối oan

ĐỀ XUẤT: 3 vùng điểm: duyệt tự động / thẩm định thủ công / từ chối.
Cân nhắc lợi nhuận từ khách tốt khi chốt ngưỡng cuối cùng.

YẾU TỐ CHÍNH: tình trạng tài khoản thanh toán > lịch sử tín dụng > thời hạn vay
```

```
> số tiền vay.

GIỚI HẠN: chỉ 1.000 hồ sơ (dữ liệu Đức thập niên 1990); kết quả dao động
±0,04 AUC giữa các lần chia; cần kiểm tra công bằng với các cột giới tính,
quốc tịch trước khi dùng.
```

---

6. Mô phỏng phỏng vấn: bảo vệ dự án 🎤

Luyện trả lời thành tiếng, mỗi câu dưới 1 phút.

1. "*Vì sao em bỏ 814 khu có giá 500.001? Như vậy có phải 'làm đẹp' số liệu không?*"
2. "*MAE test 26,4 thấp hơn MAE CV 27,3. Có phải em đã chỉnh mô hình dựa trên tập test?*"
3. "*Mô hình tín dụng của em từ chối 58% khách. Em có thật sự khuyên ngân hàng dùng nó không?*"
4. "*Nếu chỉ được chọn một: AUC hay chi phí, em báo cáo cái nào cho giám đốc?*"
5. "*Làm sao em biết mô hình không phân biệt đối xử với người nước ngoài?*"

Gợi ý trả lời

> Mẹo Notion: chọn tiêu đề "Gợi ý trả lời" → `Turn into → Toggle heading 3`.

1. "Giá 500.001 là giá bị chặn trần, giá thật không biết. Giữ lại thì mô hình học sai rằng giá không vượt 500.001. Em đã bỏ và **ghi rõ giới hạn**: mô hình không dùng cho nhà trên 500.000 USD. Không ẩn giấu, mà thu hẹp phạm vi một cách minh bạch."
2. "Không. Em chọn mọi thứ bằng cross-validation trên train, test chỉ dùng một lần ở cuối. Chênh 0,9 nằm trong dao động bình thường; mô hình cuối được huấn luyện trên toàn bộ train (nhiều dữ liệu hơn mỗi fold) nên tốt hơn một chút là hợp lý."
3. "Không dùng nguyên trạng. Ma trận chi phí 5:1 chưa tính lợi nhuận từ khách tốt. Em đề xuất chia 3 vùng điểm, vùng giữa chuyển thẩm định viên, và tính lại ngưỡng khi có số liệu lợi nhuận thật."
4. "Chi phí, vì nó nói bằng ngôn ngữ của giám đốc: tiền. AUC dùng nội bộ để so sánh mô hình trước khi chọn ngưỡng."
5. "Em chưa thể khẳng định. Cần đo các chỉ số theo từng nhóm (tỉ lệ từ chối, recall theo nhóm người nước ngoài/bản địa), thử bỏ các cột nhạy cảm và xem kết quả thay đổi thế nào. Đó là bước bắt buộc trước khi triển khai."

---

7. Đưa lên portfolio

1. Một repo GitHub `project-1-ml-co-dien` gồm: 2 notebook (A, B), file `README.md` (tóm tắt từ báo cáo mục 5, kèm 2–3 biểu đồ), file `requirements.txt` (bài 0.2).
2. README nên trả lời được trong 30 giây: **bài toán gì, dữ liệu gì, kết quả bao nhiêu so với mốc, giới hạn gì**.
3. Không đưa file dữ liệu lớn hay file `.joblib` lên Git (bài 0.2, `.gitignore`).

---

8. Flashcard tổng kết Giai đoạn 2

Mặt trước	Mặt sau
Quy trình chuẩn một dự án ML?	Chia test → đặc trưng → pipeline → so sánh bằng CV → tinh chỉnh → test 1 lần → đóng gói.
Mô hình mạnh nhất cho dữ liệu bảng thường là?	Gradient Boosting (LightGBM/XGBoost).
Khi nào ưu tiên Logistic/Linear Regression?	Dữ liệu ít, cần giải thích từng hệ số.
Chọn ngưỡng phân loại dựa vào gì?	Chi phí thật của từng loại lỗi, tính bằng CV trên train.
`cross_val_predict` dùng để làm gì?	Lấy dự đoán "như dữ liệu mới" cho mọi mẫu train, ví dụ để chọn ngưỡng.
Phân tích lỗi để làm gì?	Biết mô hình sai ở đâu, cảnh báo người dùng, tìm hướng cải thiện.
Hiện tượng "kéo về trung bình"?	Mô hình đoán cao giá trị thấp, đoán thấp giá trị cao.
Một báo cáo ML tốt phải có gì?	Kết quả so với mốc, diễn giải bằng lời, giới hạn, đề xuất.
Vì sao không dùng lambda trong pipeline cần lưu?	joblib/pickle không lưu được lambda.
Kiểm tra công bằng của mô hình thế nào?	Đo chỉ số theo từng nhóm nhạy cảm; thử bỏ cột nhạy cảm.

🎉 Hoàn thành Giai đoạn 2!

Bạn đã đi qua toàn bộ "hộp đồ nghề" ML cổ điển: hồi quy, phân loại, phân cụm, giảm chiều; từ Linear Regression đến LightGBM; và quan trọng hơn: **quy trình làm ML đúng cách**. Đây là nền móng cho mọi thứ phía sau.

Giai đoạn tiếp theo: Giai đoạn 3 – Deep Learning nền tảng. **Bài tiếp theo:** 3.1 – Perceptron: nơ-ron nhân tạo đầu tiên.